

1 SRC - Serial Roulette Controller

O módulo *Serial Roulette Controller* (SRC) é responsável pela receção, decodificação e entrega de comandos ao *Roulette Display*, sendo a interface principal entre o sistema de controlo (*Control*) e o que é apresentado na roleta.

A comunicação entre o *Control* e o SRC é realizada em série, minimizando a complexidade de interligação. A estrutura interna do SRC é composta por dois blocos principais, como apresentado na figura 1:

- **Serial Receiver** – responsável pela receção e validação de tramas; Dada a sua natureza genérica, este bloco foi também utilizado na construção do SLCDC, com as devidas adaptações;
- **Roulette Dispatcher** – responsável por atuar sobre o *Roulette Display* conforme o comando recebido. Dada a sua natureza genérica, este bloco foi também utilizado na construção do SLCDC, com as devidas adaptações;

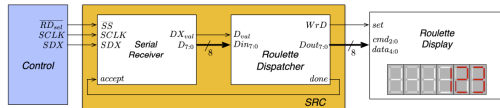


Figura 1: Diagrama de Blocos do SRC

1.1 Protocolo de Comunicação

Cada trama recebida pelo SRC contém 9 bits, distribuídos da seguinte forma 2:

- **Bits 0–2:** código de comando para o *Roulette Display*;
- **Bits 3–7:** campo de dados associado ao comando;
- **Bit 8:** bit de paridade ímpar para verificação de integridade.

A receção da trama é iniciada com uma transição descendente do sinal **notSS**, sendo os bits lidos nas transições ascendentes do sinal **SCLK**. O sinal **SDX** transporta os dados em série.

Tabela 1: Operações no *Roulette Display*

Cmd			data						Function
2	1	0	4	3	2	1	0		
0	0	0	d_4	d_3	d_2	d_1	d_0		update digit 0
0	0	1	d_4	d_3	d_2	d_1	d_0		update digit 1
...			...						...
1	0	1	d_4	d_3	d_2	d_1	d_0		update digit 5
1	1	0	–	–	–	–	–		update display
1	1	1	–	–	–	–	0		display on
1	1	1	–	–	–	–	1		display off

Figura 2: Tabela de operações

1.2 Serial Receiver

Este bloco reconstrói a trama recebida bit a bit, armazenando os dados num registo de deslocamento. Após a leitura dos 9 bits, o último bit é utilizado para verificar a paridade dos 8 bits anteriores, devendo a paridade ser ímpar.

Quando uma trama válida é detetada, os 8 bits de dados são disponibilizados ao *Roulette Dispatcher*, e o sinal **Dval** é ativado. O Serial Receiver apenas fica pronto para nova receção após a ativação do sinal **done**, sinalizado pelo *Roulette Dispatcher*.

Este comportamento segue uma arquitetura idêntica ao Serial Receiver do módulo SLCDC, mas adaptada para lidar com tramas de 9 bits em vez de 6.

Assim, tanto os componentes internos do Serial Receiver, quanto a sua máquina de estados mantêm-se idênticos, desde que se adaptem os bits como, por exemplo, para a dFlag e pFlag do controlador, mantendo todos os 4 sub-blocos como mostrado na figura 3.

- **Serial Control** — máquina de estados que gere a receção (ver Figura 4);
- **Shift Register** — armazena os bits recebidos;
- **Counter** — conta os bits recebidos;
- **Parity Check** — verifica se a paridade da trama é ímpar.

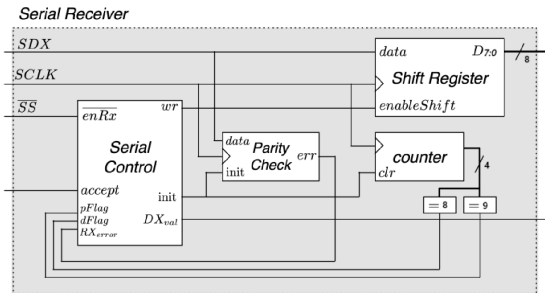


Figura 3: Diagrama de blocos do Serial Receiver

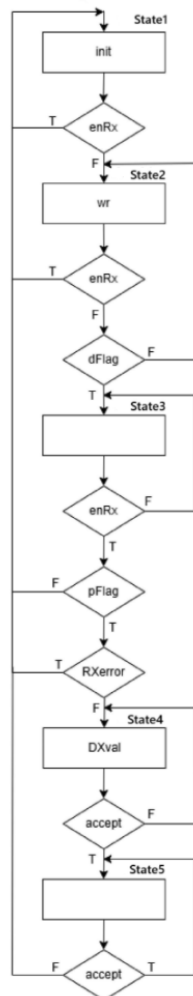


Figura 4: Máquina de estados do Serial Receiver

1.3 Roulette Dispatcher

Após a receção de uma trama válida, este bloco interpreta o comando e executa a ação correspondente sobre o *Roulette Display*.

Os comandos possíveis encontram-se resumidos na Tabela 2, com base nos 3 primeiros bits da trama.

O campo de dados de 5 bits é utilizado como argumento para a operação (por exemplo, o valor a mostrar).

Uma vez que o comando foi entregue ao *Roulette Display*, o sinal **done** é ativado, permitindo ao *Serial Receiver* iniciar a receção da próxima trama.

Este comportamento segue uma arquitetura idêntica ao LCD Dispatcher do módulo SLCDC, mas adaptada para lidar com tramas de 8 bits em vez de 5.

Além disso, ao contrário do Serial Receiver, que precisava de adaptar certos valores, para fazer comparações internas, o dispatcher tem exatamente o mesmo comportamento que o dispatcher do SLCDC, pelo que foi apenas feita a mudança de bits acima mencionada.

Assim, a máquina de estados do dispatcher está representada na figura 5

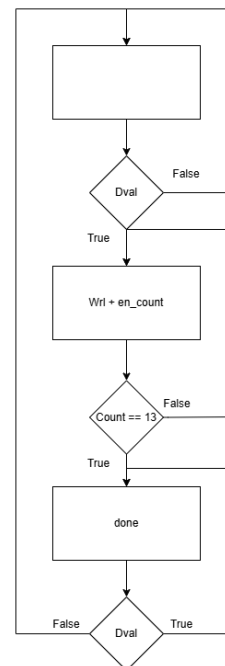


Figura 5: Máquina de estados do Roulette Dispatcher

2 Control - Software

A componente de *Control*, desenvolvida em Kotlin, implementa a lógica de alto nível do sistema de jogo da roleta, interagindo com os periféricos (teclado, LCD, display da roleta) através de módulos especializados. Esta arquitetura modular e bem separada permite manter o código coeso, testável e facilmente extensível.

2.1 SerialEmitter

O **SerialEmitter** é o módulo responsável por enviar tramas codificadas através da interface série, usadas para comunicar com os módulos SLCDC (LCD) e SRC (roleta).

Cada trama contém:

- Bits de dados (4 ou 8 bits conforme destino);
- Bit RS ou comando (LCD ou Roleta);
- Bit de paridade ímpar.

As funções principais incluem:

- **send(destination, data, size)** — envia a trama codificada com 1 bit de paridade ímpar;
- **init()** — inicializa ambos os destinos e limpa a linha de clk;
- **pulseClock()** - impõe um ciclo de SCLK clock;

2.2 RouletteDisplay

O módulo **RouletteDisplay** é responsável por controlar o display da roleta, através de tramas enviadas ao componente SRC.

Comandos possíveis:

- **init()** — Inicialização do Serial Emitter e ligação dos displays;
- **setValue(value: Int)** e **setValue(value: String)** — apresenta um valor, escrito em string ou inteiro;
- **animation()** — inicia a animação do sorteio;
- **off(enabled: Boolean)** — ativa ou desativa o display.

- **on()** e **off()** — chama a função anterior, com o valor booleano predefinido de acordo com o nome;

As tramas seguem o formato de 8 bits (3 de comando, 5 de dados).

2.3 TUI - Text User Interface

O objetivo do TUI é permitir abstrair a interação com o utilizador final, através do LCD e teclado. Para isso ele deve fornecer utilitários para apresentar menus, pedir inputs, mostrar mensagens, limpar o ecrã, entre outros.

Este módulo serve de ponte entre a lógica do jogo e os periféricos visuais e de input.

Para isso, foram implementadas as funções:

- **init()** — Inicialização do LCD e do KBD;
- **writeMessage()** - Escrita de uma string
- **getKey()** - Guarda o valor da key premida;
- **isMaintenance()** - Verifica se o sistema está no modo de manutenção.